

Student Performance Analyzer

A simple command-line based Student Performance Analyzer in Python. The project allows the user to manage student information, enter marks and attendance, generate individual student reports and analyze overall class performance.

Student Details

Name: Manas Pandey

Registration ID: 26BET10001

Branch: CSE – Educational Technology

Project Type: Python Console Application

1. Project Overview

The Student Performance Analyzer is a Python console application that helps to manage and analyze the academic information of students.

The application stores the student details in Python dictionaries and provides features like

Adding new students

Adding subject-wise marks

Records attendance

Generating an individual student reports

Displaying all the students

Analyzing the class performance

Calculating grades and performance levels

The project is made as a beginner level project to show the basic Python programming concepts.

2. Features

Add Student

It helps the user to add the following information

Roll number

Student name

Branch

Semester

Every student is added with the roll number as the unique key for identification purposes.

Add Marks

It helps the user

To select a student by roll number

To add the number of subjects

To enter the subjects and marks of each subject for the selected student. The marks entered should be a number between 0 and 100.

Add Attendance

It helps the user to

Enter total number of classes

Enter the number of classes attended

The application then calculates the percentage of attendance.

Student Report

It generates a student report which displays the following information

Student information

Subject-wise marks

Total marks

Average percentage

Grade

Attendance percentage

Attendance status

The best subject

The weakest subject

Overall performance

Display Students

The application helps the user to view all the students along with the following information

Roll number

Name

Branch

Performance Analysis

It helps to analyze the students for whom the marks have been added and calculate the following

Highest performing student

Lowest performing student

Average for the class

3. Grade System

The grade is calculated based on the percentage of the total marks with the following system

Percentage Grade

90-100 A+

80-89 A

70-79 B+

60-69 B

50-59 C

40-49 D

Below 40 F

4. Performance System

Based on the percentage of marks, the performance is calculated with the following system

Percentage Performance

75% and above Excellent

60%-74% Good

50%-59% Average

40%-49% Needs Improvement

Below 40% Poor

5. Attendance System

Attendance percentage is calculated with the below formula $\text{Attendance Percentage} = (\text{Classes Attended} / \text{Total Classes}) \times 100$ The attendance status is displayed along with attendance percentage as shown below

Good: If the attendance is $\geq 75\%$

Warning: If the attendance is $< 75\%$

6. Technologies Used

Programming Language: Python

Interface: Command Line Interface (CLI)

Data Storage: Python dictionaries

Development Environment: Visual Studio Code

The application does not have any external Python libraries.

7. Python Concepts Used

This project demonstrates several Python concepts and some of the concepts used are

Variables

Dictionaries

Nested dictionaries

Functions

Conditional statements

if, elif and else

while loops

for loops

Taking user input through input()

Type conversion

Exception handling with try-except

Dictionary methods

sum()

len()

max()

min()

Lambda functions

Formatted strings

String formatting

8. Project Structure

Student-Performance-Analyzer/

|

|— student_performance_analyzer.py

|

|— README.md

9. Requirements

The below prerequisites need to be fulfilled for the project

Python 3.x

A terminal to run the project

A code editor like Visual Studio Code

10. Installation and Setup

First, Install Python

Install Python 3.x on your system.

Verify if the Python is installed correctly by running the below commands in the terminal

If it is Python 3, run the below command

`python --version`

Else, if it is Python 2, run the below command

`python3 --version`

Second, clone the git repository

Use the below command to clone the repository

`git clone https://github.com/your-username/your-repository-name.git`

Move into the project directory with the below command

`cd your-repository-name`

Third, run the program

Run the below command to run the program

`python student_performance_analyzer.py`

If your system uses Python 3, run the command

`python3 student_performance_analyzer.py`

11. How to Use the Application

Once the application is started, you will see the below command-line menu

STUDENT PERFORMANCE ANALYZER

1. Add Student
2. Add Marks
3. Add Attendance
4. Student Report
5. Display Students
6. Performance Analysis
7. Exit

Enter the corresponding option number to perform an action.

Recommended way to use the application is as below

1. Choose 1 to add a student.
2. Choose 2 to add the subject marks.
3. Choose 3 to add attendance.
4. Choose 4 to display the student report.
5. Choose 5 to view the list of all the students.
6. Choose 6 to analyze the class performance.
7. Choose 7 to exit the application.

12. Data Structure

The project uses nested Python dictionaries to store the information and a sample data structure is shown below.

```
students = {  
    "101": {  
        "name": "Student Name",  
        "branch": "CSE",  
        "semester": "1",  
        "subjects": {},
```

```
"total": 0,  
"attended": 0  
}  
}
```

The roll number is used as a key to store and retrieve the student information. All the subject marks are stored inside the subjects dictionary.

13. Input Validation

The below basic validations are implemented to improve the reliability of the application.

Duplicate roll numbers are not added.

Marks entered should be between 0 and 100.

Any invalid number entry is handled with try-except.

Attendance cannot be more than the total classes.

Negative attendance is not allowed.

Report cannot be displayed for a non-existing student.

14. Limitations

There are a few limitations with the current setup.

All the information is stored in memory and it is lost once the application is closed since it does not have any permanent storage like a database or a file system. It is also a command line application and does not have a login system. The marks entered for the students disappear once the application is exited.

The limitations can be resolved in the future releases.

15. Future Enhancements

Some of the possible enhancements are

Data can be permanently stored using files or databases like CSV, JSON, SQLite etc., and provide features to edit or delete records.

Provide search functionality.

Analyze the performance individually for each subject.

Provide GUI support.

Provide login/logout or authentication to secure the application.

Export student report in PDF or any other preferred format.

Improve the error handling mechanism.

Analyze the class-wise and semester-wise performance etc.

16. Learning Outcomes

With this project, we have the below ten learning outcomes related to programming skills.

1. Designing menu driven applications in Python.
2. Using of dictionaries to store and manage the structured data.
3. Using functions to avoid code duplication.
4. Using of conditional statements.
5. Using loops.
6. Input validation.
7. Error and exception handling.
8. Performing mathematical calculations.
9. Using built-in functions.
10. Performing data analysis related calculations.

17. Conclusion

The Student Performance Analyzer is a basic Python application that shows how the basic programming concepts can be used to develop a useful academic management application and the project can be extended to a fully-fledged student management and academic analysis application with databases, GUI and permanent storage in future.

Author

Manas Pandey

Registration ID: 26BET10001

Branch: CSE – Educational Technology